Foundations of Neural style transfer

[Neural style transfer]

1.0 Content Block

Minimizing this loss encourages the generated image to have similar style characteristics to the style image, as captured by the correlations between feature responses in each layer. The idea is that activation pattern correlations between filters in a single layer captures the "style" on the order of the receptive fields at that layer. Similarly to the previous case, the w l {\displaystyle w_{l}} are positive real numbers chosen as hyperparameters.

2.0 Content Block

E l = 1 4 N l 2 M l 2 ∑ i , j (G i j l (x →) − G i j l (a →)) 2 . {\displaystyle E_{l}={\frac {1}{4N_{l}^{2}M_{l}^{2}}}\sum _{i,j}\left(G_{ij}^{l}({\vec {x}})-G_{ij}^{l}({\vec {a}})\right)^{2}.} Here, G i j l (x →) {\displaystyle G_{ij}^{l}({\vec {x}})} and G i j l (a →) {\displaystyle G_{ij}^{l}({\vec {a}})} are the entries of the Gram matrices for the generated and style images at layer l {\displaystyle l} . Explicitly, G i j l (x →) = ∑ k F i k l (x →) F j k l (x →) {\displaystyle G_{ij}^{l}({\vec {x}})=\sum _{k}F_{ik}^{l}({\vec {x}})F_{jk}^{l}({\vec {x}})}

3.0 Content Block

F i j l (x →) {\textstyle F_{ij}^{l}({\vec {x}})} is the activation of the i th {\textstyle i^{\text{th}}} filter at position j {\textstyle j} in layer l {\textstyle l} . A given input image x → {\textstyle {\vec {x}}} is encoded in each layer of the CNN by the filter responses to that image, with higher layers encoding more global features, but losing details on local features.

4.0 Content Block

Neural style transfer (NST) software algorithms are able to manipulate digital images, or videos, in order to adopt the appearance or visual style of another image. NST algorithms are characterized by their use of deep neural networks for the sake of image transformation. Common uses for NST are the creation of artificial artwork from photographs, for example by transferring the appearance of famous paintings to user-supplied photographs. Several notable mobile apps use NST techniques for this purpose, including DeepArt and Prisma. This method has been used by artists and designers around the globe to develop new artwork based on existent style(s).

5.0 Content Block

Hyperparameters In the original paper, they used a particular choice of hyperparameters. The style loss is computed by w l = 0.2 {\displaystyle w_{l}=0.2} for the outputs of layers conv1_1, conv2_1, conv3_1, conv4_1, conv5_1 in the VGG-19 network, and zero otherwise. The content loss is computed by w l = 1 {\displaystyle w_{l}=1} for conv4_2, and zero otherwise. The ratio α / β ∈ [5 , 50] × 10 − 4 {\displaystyle \alpha /\beta \in [5,50]\times 10^{-4}} .

6.0 Content Block

Style loss The style loss is based on the Gram matrices of the generated and style images, which capture the correlations between different filter responses at different layers of the CNN:

7.0 Content Block

Symbols Let x → {\textstyle {\vec {x}}} be an image input to a CNN. Let F l ∈ R N l × M l {\textstyle F^{l}\in \mathbb {R} ^{N_{l}\times M_{l}}} be the matrix of filter responses in layer l {\textstyle l} to the image x →

{\textstyle {\vec {x}}} , where:

8.0 Content Block

History NST is an example of image stylization, a problem studied for over two decades within the field of non-photorealistic rendering. The first two example-based style transfer algorithms were image analogies and image quilting. Both of these methods were based on patch-based texture synthesis algorithms. Given a training pair of images–a photo and an artwork depicting that photo–a transformation could be learned and then applied to create new artwork from a new photo, by analogy. If no training photo was available, it would need to be produced by processing the input artwork; image quilting did not require this processing step, though it was demonstrated on only one style. NST was first published in the paper "A Neural Algorithm of Artistic Style" by Leon Gatys et al., originally released to ArXiv 2015, and subsequently accepted by the peer-reviewed CVPR conference in 2016. The original paper used a VGG-19 architecture that has been pre-trained to perform object recognition using the ImageNet dataset. In 2017, Google AI introduced a method that allows a single deep convolutional style transfer network to learn multiple styles at the same time. This algorithm permits style interpolation in real-time, even when done on video media.

9.0 Content Block

Overview The idea of Neural Style Transfer (NST) is to take two images—a content image p → {\displaystyle {\vec {p}}} and a style image a → {\displaystyle {\vec {a}}} —and generate a third image x → {\displaystyle {\vec {x}}} that minimizes a weighted combination of two loss functions: a content loss L content (p → , x →) {\displaystyle {\mathcal {L}}_{\text{content }}({\vec {p}},{\vec {x}})} and a style loss L style (a → , x →) {\displaystyle {\mathcal {L}}_{\text{style }}({\vec {a}},{\vec {x}})} . The total loss is a linear sum of the two: L NST (p → , a → , x →) = α L content (p → , x →) + β L style (a → , x →) {\displaystyle {\mathcal {L}}_{\text{NST}}({\vec {p}},{\vec {a}},{\vec {x}})=\alpha {\mathcal {L}}_{\text{content}}({\vec {p}},{\vec {x}})+\beta {\mathcal {L}}_{\text{style}}({\vec {a}},{\vec {x}})} By jointly minimizing the content and style losses, NST generates an image that blends the content of the content image with the style of the style image. Both the content loss and the style loss measures the similarity of two images. The content similarity is the weighted sum of squared-differences between the neural activations of a single convolutional neural network (CNN) on two images. The style similarity is the weighted sum of Gram matrices within each layer (see below for details). The original paper used a VGG-19 CNN, but the method works for any CNN.

10.0 Content Block

Training Image x → {\displaystyle {\vec {x}}} is initially approximated by adding a small amount of white noise to input image p → {\displaystyle {\vec {p}}} and feeding it through the CNN. Then we successively backpropagate this loss through the network with the CNN weights fixed in order to update the pixels of x → {\displaystyle {\vec {x}}} . After several thousand epochs of training, an x → {\displaystyle {\vec {x}}} (hopefully) emerges that matches the style of a → {\displaystyle {\vec {a}}} and the content of p → {\displaystyle {\vec {p}}} . As of 2017, when implemented on a GPU, it takes a few minutes to converge.